EXPERIMENT - 9

Railway Booking Indexing and Query Performance

AIM

To create a Railway Booking table containing at least 5,000 records, benchmark query performance before and after creating single and composite indexes, and compare the execution plans using EXPLAIN.

1. CREATE TABLE AND GENERATE 5,000 RECORDS

```
CREATE DATABASE IF NOT EXISTS Experiment9;
USE Experiment9;
CREATE TABLE Railway_Booking (
 Booking_ID INT PRIMARY KEY, PNR VARCHAR(15) UNIQUE,
 Passenger_Name VARCHAR(60), Train_No INT, Train_Name VARCHAR(80),
 Journey_Date DATE, Source_Station VARCHAR(50),
 Destination_Station VARCHAR(50), Class_Type VARCHAR(20),
 Fare DECIMAL(10,2), Booking_Status VARCHAR(20)
) ENGINE=InnoDB;
SET SESSION cte_max_recursion_depth=6000;
INSERT INTO Railway_Booking
(Booking_ID,PNR,Passenger_Name,Train_No,Train_Name,Journey_Date,
 Source_Station,Destination_Station,Class_Type,Fare,Booking_Status)
WITH RECURSIVE seq AS (
 SELECT 1 AS n UNION ALL SELECT n+1 FROM seq WHERE n<5000
)
SELECT n, CONCAT('PNR',LPAD(n,8,'0')), CONCAT('Passenger_',n),
 CASE MOD(n,5) WHEN 0 THEN 12001 WHEN 1 THEN 12002
 WHEN 2 THEN 12003 WHEN 3 THEN 12004 ELSE 12005 END,
 CASE MOD(n,5) WHEN 0 THEN 'Shatabdi Express'
 WHEN 1 THEN 'Rajdhani Express' WHEN 2 THEN 'Duronto Express'
 WHEN 3 THEN 'Gomti Express' ELSE 'Intercity Express' END,
 DATE_ADD('2026-11-01',INTERVAL MOD(n,30) DAY),
 CASE MOD(n,4) WHEN 0 THEN 'Delhi' WHEN 1 THEN 'Lucknow'
 WHEN 2 THEN 'Kanpur' ELSE 'Varanasi' END,
 CASE MOD(n,4) WHEN 0 THEN 'Lucknow' WHEN 1 THEN 'Delhi'
 WHEN 2 THEN 'Varanasi' ELSE 'Prayagraj' END,
 CASE MOD(n,3) WHEN 0 THEN 'AC 2 Tier'
 WHEN 1 THEN 'AC 3 Tier' ELSE 'Sleeper' END,
 500+MOD(n,20)*75,
 CASE MOD(n,4) WHEN 0 THEN 'CONFIRMED' WHEN 1 THEN 'WAITING'
 WHEN 2 THEN 'RAC' ELSE 'CANCELLED' END
FROM seq;
SELECT COUNT(*) AS Total_Bookings FROM Railway_Booking;
```

OUTPUT: VERIFY RECORD COUNT

Total_Bookings
5000

2. BENCHMARK BEFORE INDEXING

```
EXPLAIN
SELECT * FROM Railway_Booking WHERE Train_No=12003;
EXPLAIN ANALYZE
SELECT * FROM Railway_Booking WHERE Train_No=12003;
```

OUTPUT

EXPLAIN shows the optimizer plan; EXPLAIN ANALYZE reports actual execution details for this MySQL server.

3. CREATE SINGLE INDEX

```
CREATE INDEX idx_train_no ON Railway_Booking(Train_No);
SHOW INDEX FROM Railway_Booking;
EXPLAIN
SELECT * FROM Railway_Booking WHERE Train_No=12003;
EXPLAIN ANALYZE
SELECT * FROM Railway_Booking WHERE Train_No=12003;
```

OUTPUT: INDEX VERIFICATION

Index	Column	Type
idx_train_no	Train_No	Non-unique secondary index

The EXPLAIN and EXPLAIN ANALYZE result sets show the access path, estimated rows, actual rows, and timing selected/reported by the server.

4. CREATE COMPOSITE INDEX

```
CREATE INDEX idx_train_date
ON Railway_Booking(Train_No, Journey_Date);
EXPLAIN
SELECT * FROM Railway_Booking
WHERE Train_No=12003
AND Journey_Date BETWEEN '2026-11-05' AND '2026-11-15';
EXPLAIN ANALYZE
SELECT * FROM Railway_Booking
WHERE Train_No=12003
AND Journey_Date BETWEEN '2026-11-05' AND '2026-11-15';
```

OUTPUT: COMPOSITE INDEX

Index	Column order
idx_train_date	Train_No, Journey_Date

The composite index supports the equality condition on Train_No followed by the Journey_Date range.

5. COMPARE SINGLE AND COMPOSITE INDEX USAGE

```
EXPLAIN
SELECT Booking_ID, Passenger_Name, Journey_Date
FROM Railway_Booking WHERE Train_No=12003;

EXPLAIN
SELECT Booking_ID, Passenger_Name, Journey_Date
FROM Railway_Booking
WHERE Train_No=12003
AND Journey_Date BETWEEN '2026-11-05' AND '2026-11-15';
```

OUTPUT: PLAN COMPARISON

Query	Filter	Index to inspect
Single-column	Train_No = 12003	idx_train_no
Composite	Train_No and Journey_Date range	idx_train_date

6. PERFORMANCE COMPARISON

```
SELECT 'BEFORE INDEX' AS Stage,
 'Use the EXPLAIN ANALYZE result recorded above' AS Observation
UNION ALL
SELECT 'SINGLE INDEX',
 'Compare actual time and rows after idx_train_no'
UNION ALL
SELECT 'COMPOSITE INDEX',
 'Compare actual time and rows after idx_train_date';
```

OUTPUT

Stage	Observation
BEFORE INDEX	Use the EXPLAIN ANALYZE result recorded above
SINGLE INDEX	Compare actual time and rows after idx_train_no
COMPOSITE INDEX	Compare actual time and rows after idx_train_date

RESULT

The dataset contains 5,000 railway bookings. The queries compare the unindexed Train_No filter with the single-index and composite-index plans. Record the actual timing and row counts reported by EXPLAIN ANALYZE on the target MySQL server; these values depend on its version, statistics, and host.